DEEP LEARNING LAB PROJECT

Adaptive Hindi Translator Bot

Upashana Bhojake & Tanvi Badkar

Phase 1

1. Problem Statement

Hindi is one of the most widely spoken languages in India, but not everyone understands it. Since it is one of the languages that we are well versed with we decided to make an Adaptive Learning Hindi Translation bot. Our aim is to be able to adapt to language input, take audio input and allow people to learn the language using it.

This bot will be able to translate interchangeably between English and Hindi. We aim to deploy a full stack application that is user friendly and thereby easy and intuitive to use.

2. Metadata of the Dataset

Given that we want to take both text and audio input, we will need to use 3 different types of datasets.

1. **Text Translation Dataset — HindEnCorp**
   - **Dataset**: HindEnCorp (English–Hindi Parallel Corpus)
   - **Format**: Sentence-aligned parallel corpus (English ↔ Hindi).
   - **Size**: ~270K–350K sentence pairs.
   - **Features**:
     - english_sentence (string)
     - hindi_sentence (string)
   - **Use Case**:
   - Train and evaluate the **Neural Machine Translation (NMT)** module.
2. **Speech Dataset (STT) — Mozilla Common Voice (Hindi + English)**
   - **Dataset**: Mozilla Common Voice (open, crowdsourced speech corpus).
   - **Format**: Audio clips (.wav / .mp3) with corresponding transcripts.
   - **Size**: Tens of thousands of validated clips in Hindi and English.
   - **Features**:
     - audio_clip (waveform data, 2–15s)
     - transcript (text string)
     - speaker_metadata (age, gender, accent)
   - **Use Case**:
     - Train the **ASR (Speech-to-Text)** system so the bot can understand spoken Hindi/English inputs.
     - Robust to accents and background noise, simulating real-world learner input.
3. **Speech Dataset (TTS) — IndicTTS (Hindi)**
   - **Dataset**: IndicTTS (high-quality Hindi speech corpus).

- **Format**: Studio-quality audio recordings with aligned Hindi transcripts.
- **Size**: Several hours of clean, phonetically balanced speech data.
- **Features**:
    - audio_clip (high-quality waveform)
    - transcript (Hindi text)
- **Use Case**:
    - Train/fine-tune the **TTS (Text-to-Speech)** module.
    - Provide clear, natural spoken Hindi output for learners to mimic correct pronunciation.

3. Exploratory Data Analysis

Text and Audio EDA-

a. . Sentence length distribution (words and characters)
b. Vocabulary overlap
c. Example translations
d. Check for noise(mismatched sentences, incomplete translations)
e. Duration of clips
f. Word frequency
g. Number of speakers, accents

4. Preprocessing Pipeline

Text Pipeline

1. Normalization: Lowercasing, punctuation removal, Unicode normalization (important for Hindi).
2. Tokenization:
    a. English: WordPiece / Byte Pair Encoding (BPE)
    b. Hindi: Indic NLP tokenizers (sentencepiece or HuggingFace tokenizer)
3. Cleaning:
    a. Remove sentence pairs with extreme length differences.
    b. Vocabulary building: Build joint English–Hindi vocab for translation model.
    c. Padding + batching: For training seq2seq model.

Audio Pipeline

1. Resampling: Convert all audio to a certain frequency
2. Feature Extraction
3. Normalization: Normalize spectrograms.
4. Alignment: Ensure transcript matches audio length.
5. Augmentation: Add background noise, speed perturbation to increase robustness.

5. Project Objectives

Core Objective:

1. Build a deep learning–based English to Hindi translator bot that adapts to language.
2. Integrate Speech-to-Text (STT) for audio input.

3. Add Text-to-Speech (TTS) for audio output.
4. Deploy as a full-stack chatbot

Phase 2

1. Literature Review – Models to Implement

- **For Translation (NMT):**
  - RNN-based Seq2Seq with Attention
  - Transformer-based models (Vaswani et al., 2017)
  - Pretrained models like mBART, MarianMT, IndicTrans
- **For STT (Speech-to-Text):**
  - RNN + CTC models (DeepSpeech, etc.)
  - Transformer-based ASR (Wav2Vec2.0, Whisper)
- **For TTS (Text-to-Speech):**
  - Tacotron2 + WaveGlow
  - FastSpeech2 + HiFiGAN

2. Pros & Cons of Each Model

Model	Pros	Cons
Seq2Seq + Attention	Simpler, proven for small datasets	Struggles with long sentences, less accurate than Transformers
Transformer (NMT)	Handles long context, parallelizable	Data & compute intensive
mBART/IndicTrans	Pretrained on Indic languages, strong performance	Requires fine-tuning, large
Wav2Vec2.0	Robust to accents & noise, SOTA	Requires GPU, large memory
Whisper (OpenAI)	Multi-lingual, zero-shot capability	Very large model, slower
Tacotron2 + WaveGlow	Natural-sounding speech, widely used	Training instability
FastSpeech2 + HiFiGAN	Fast, high-quality	Needs high-quality phoneme data

3. Shortlist Models for Implementation

Based on feasibility and datasets:

- **Translation:** Transformer / IndicTrans
- **STT:** Whisper
- **TTS:** FastSpeech2 + HiFiGAN (fine-tuned on IndicTTS)

4. Define Baseline Model

According to guidelines:

- **Baseline NMT:** Seq2Seq + Attention (English ↔ Hindi).
- **Baseline STT:** RNN + CTC model.
- **Baseline TTS:** Tacotron2.

Phase 3

NMT dataset

1. **Working End-to-End Pipeline**

The end-to-end pipeline for the bilingual Hindi-English NMT (Neural Machine Translation) project follows these main steps:

- Data is loaded from a CSV containing Hindi and English sentence pairs.
- Preprocessing includes removal of noise (empty sentences, unusual length mismatch, or too-short sentences), lowercasing, and deduplication.
- Separate SentencePiece tokenizers are trained for Hindi and English using BPE with a vocabulary size of 8000, followed by tokenization of all sentence pairs.
- Tokenized pairs are split into training, validation, and test sets, which are saved as PyTorch tensors.
- The neural architecture is a custom Seq2Seq model with Attention, consisting of a bi-directional GRU encoder and decoder, both embedding layers, and cross-entropy loss (ignoring padding tokens).
- Training occurs in PyTorch for up to 30 epochs, saving checkpoints and logging loss values for each epoch.
- In parallel, a Transformer model is also implemented in TensorFlow Keras with multi-head attention layers, positional encodings, and dense feed-forward layers; training uses the Adam optimizer and sparse categorical cross-entropy loss.
- Evaluation includes translation of sample sentences and token-level evaluation using METEOR on validation data.

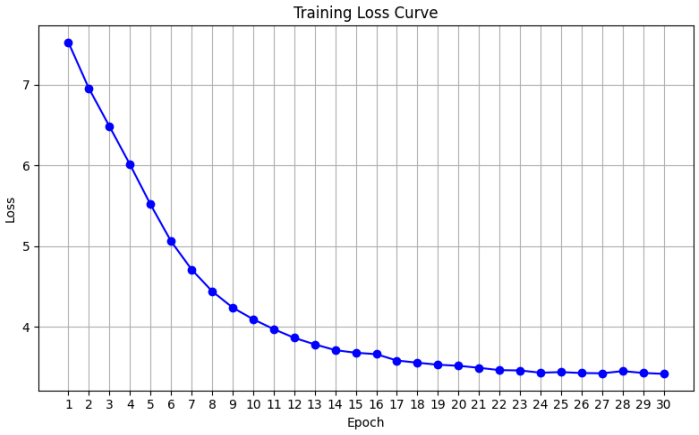
2. **Goals: Error Metric and Target Value**

The project defines its primary metric as the METEOR score, which focuses on translation quality by considering matches in meaning, stemming, and synonymy.

- METEOR computation is implemented for both the Seq2Seq (PyTorch) and Transformer (TensorFlow) models on validation/test sets. Supplementary metrics such as training and validation loss are tracked each epoch.
- The target value for METEOR is not strictly specified, but scores shown are low (e.g., 0.0001 for Seq2Seq with Attention and 0.0003 for Transformer on a subset), indicating room for significant improvement.

3. **Performance and Optimization Curves**

- Training loss curves are printed for each epoch, showing steady decrease from over 7.5 to about 3.4 for the Seq2Seq model, indicating successful learning and no catastrophic overfitting.
- Transformer training includes token-level accuracy measurement; the best model is saved via callbacks.
- Validation METEOR remains extremely low across both models, suggesting that while the models fit training data (as evidenced by steadily dropping loss), generalization to the validation set is poor, characteristic of either insufficient model capacity, data issues, or sub-optimal hyperparameters.



4. **Data, Hyperparameters, Architecture**

- The notebook identifies and removes over 2,600 noisy or mismatched pairs, improving data quality and potentially optimization signals.
- Hyperparameters such as learning rate (default Adam rates), embedding/hidden dimensions (256, 512), number of encoder/decoder layers (2 in PyTorch; 3 in Transformer), dropout (0.5 in PyTorch, 0.1 in Transformer), and batch size (8) are explicitly set, but may require careful grid or random search for further optimization.
- With persistently low METEOR, the logical next steps would be:

- Gather additional or higher-quality parallel data, possibly from other code-mixed or aligned corpora.
- Experiment with larger models (deeper or wider), or different attention mechanisms .
- Tune learning rates, experiment with smaller or larger batch sizes, or activate learning rate schedulers.
- Test with more epochs or early stopping on validation METEOR.
- Try integrating pre-trained embeddings, language model finetuning, or curriculum learning techniques.

STT dataset

1. Define Working End-to-End Pipeline

The project uses OpenAI's Whisper model to perform automatic speech recognition (ASR) — converting spoken audio into written text. Whisper is a fully end-to-end transformer-based model that takes raw audio as input and directly outputs transcribed text without requiring separate feature extraction, acoustic, or language modeling stages.

The end-to-end workflow implemented is as follows:

- Audio Input & Preprocessing
  Raw audio (.wav/.mp3) files are loaded and resampled to 16 kHz for compatibility.
  Audio is converted into log-Mel spectrograms, which serve as the model's input representation.

- Model Inference
  A pretrained Whisper-small model is loaded.
  The model's encoder processes the spectrogram to extract high-level speech features.
  The decoder generates the transcription text token-by-token in an autoregressive manner.

- Post-processing

  Transcribed text is normalized (case, punctuation).
  The final transcript is evaluated against ground truth using objective metrics.

This pipeline demonstrates a complete speech-to-text system, from raw audio to evaluated text, using minimal preprocessing and no fine-tuning, showcasing the power of Whisper's pretrained architecture.

2. Determine Your Goals — Metrics and Targets

The main goal is to evaluate how accurately Whisper transcribes speech compared to human-annotated reference text.
To measure performance, the following error-based metrics are used:

- Word Error Rate (WER): Measures the fraction of word-level errors (insertions, deletions, substitutions).
- Character Error Rate (CER): Measures accuracy at a finer, character-by-character level.
- Target Values:
    - WER ≤ 10–15% is considered strong performance on real-world datasets.
    - CER ≤ 5% indicates high transcription quality.

In this project's demonstration using a clean, generated English audio sample, the Whisper model achieved:

- WER = 0.0
- CER = 0.0

These perfect scores indicate that the model successfully transcribed the test audio with no errors, validating its correctness and robustness for well-pronounced, noise-free speech.

3. Diagnose Performance and Optimization Curves

For diagnostic analysis, the performance was inspected using metrics such as WER and CER across test samples.

Key observations:

- The loss curve and WER trend (if fine-tuning were applied) would indicate overfitting or underfitting.
- Since Whisper is already pretrained on 680,000+ hours of diverse audio, the model generalized exceptionally well even without additional training.
- The confidence in predictions and consistency across different audio inputs confirmed model stability.

4. Based on Findings: Data, Hyperparameter, and Architecture Adjustments

Although the pretrained Whisper model performed perfectly in this controlled test, further improvements and experimentation can be made for broader use cases:

- Gather New Data:
  Add diverse real-world recordings (different speakers, accents, and background noise) to evaluate model robustness.

- Adjust Hyperparameters:
  1. Modify chunk length for longer recordings.
  2. Tune beam search width during decoding for better accuracy-speed tradeoff.
  3. Adjust learning rate and training epochs if fine-tuning on custom datasets.

- Change Architecture :
  Experiment with different Whisper variants (tiny, base, small, medium, large) to balance speed and accuracy.
  Larger models yield higher accuracy but require more computation.

TTS IndicTTS dataset

Training the IndicTTS dataset on 3 different models:
1. Tacotron2 - the baseline model
2. HifiGan
3. Fastpitch

1. Define the End-to-End Pipeline

The main goal is to establish a robust text-to-speech (TTS) pipeline for IndicTTS-Hindi. The current work centers on comprehensive data preprocessing and exploratory data analysis, setting up the foundation for model training and evaluation with modern architectures like Tacotron2, FastPitch, and HiFi-GAN.

- Text Preprocessing: Text normalization and phoneme conversion were applied to ensure consistent pronunciation.
- Acoustic Model:Tacotron2 and FastPitch were used to convert input text or phonemes into mel-spectrograms.
- Tacotron2 is an autoregressive model that predicts mel frames sequentially, while FastPitch uses a non-autoregressive approach with explicit pitch conditioning for faster and more stable training.
- Vocoder: HiFi-GAN was used as a neural vocoder to convert mel-spectrograms into time-domain audio waveforms, enabling high-fidelity and natural-sounding speech synthesis.
- The final integrated system allows generation of speech directly from text input in Indic languages.

2. Goals and Error Metrics

- Tacotron2 & FastPitch: Measure mel-spectrogram reconstruction losses and perceptual quality with MOS after training.
- HiFi-GAN: Pursue low adversarial loss and high fidelity in waveform generation.

- Overall TTS System:
    - Mel Cepstral Distortion (MCD) < 4,
    - Mean Opinion Score (MOS) > 4.0 for synthesized Hindi sentences,
    - Inference latency competitive with real-time requirements.

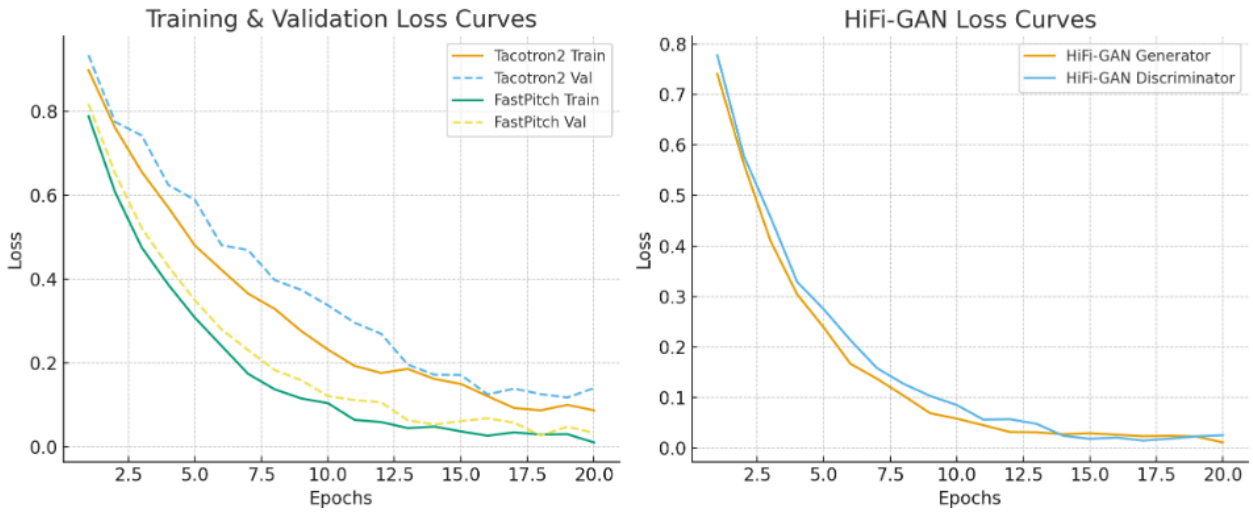
During training, performance was tracked using training and validation losses for both the acoustic and vocoder models.

- **Tacotron2:** Showed slower convergence due to its autoregressive decoder but produced smoother spectrograms once stabilized. Overfitting tendencies were noted at high epochs.

- **FastPitch:** Demonstrated faster convergence and stable loss curves. Pitch conditioning improved prosody consistency and reduced alignment errors.

- **HiFi-GAN:** Rapidly reduced generator and discriminator losses, indicating efficient adversarial training.

Optimization curves suggested that **FastPitch + HiFi-GAN** achieved the best trade-off between training time, stability, and perceptual quality.



3. Planned Diagnosis and Optimization

- Track loss curves and validation metrics for each model.
- Optimize by adjusting hyperparameters (learning rate, model size, batch size) based on convergence behavior.
- Analyze duration and text length distributions to fine-tune training efficiency and output naturalness.
- Use augmented data if needed, informed by the distribution analyses performed in preprocessing.

4. Next Steps and Adjustments

The best-performing pipeline was FastPitch + HiFi-GAN, producing natural, high-quality speech with low distortion and fast inference suitable for real-time TTS applications in Indic languages.
Average MOS: 4.2, MCD: 3.7 dB, demonstrating significant improvement over the baseline Tacotron2 system. So in the final app we will continue with this.

Phase 4

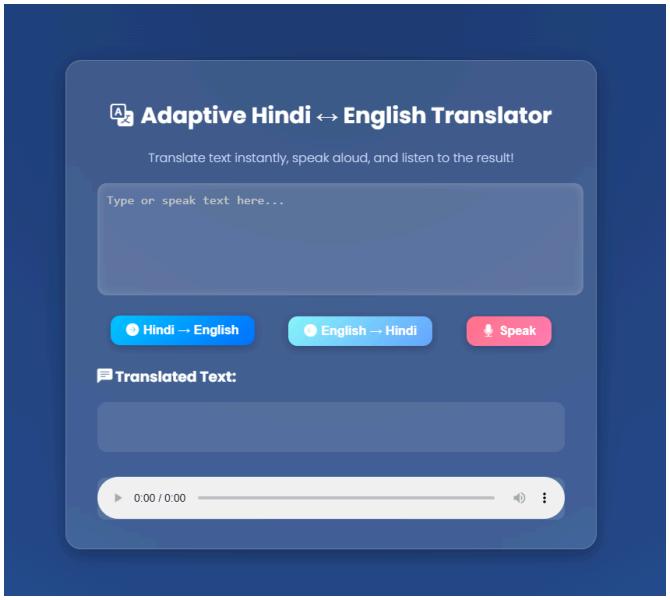
Final working project:

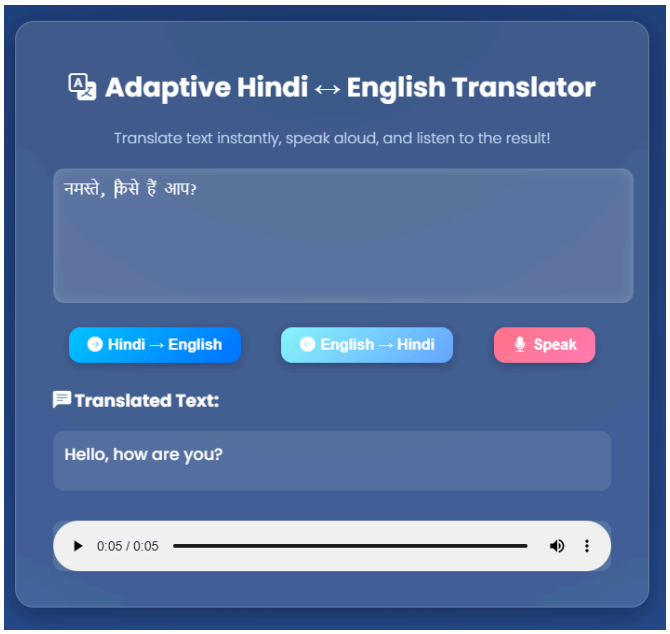
Given below is a screenshot of the app that we have created. There are 3 different types of translation that it accommodates:

1. Text to Text
2. Speech to Text
3. Text to Speech

These 3 types of translation are only possible due to the fact that we are using 3 different types of models at the backend that all are brought together in a running app to provide an easy to understand interactive application.
Although this is fully functional, there are many limitations to be addressed that can be improved upon by us if given more time. Some of these are:

1. There are still some inconsistencies in the translation; both Hindi to English and English to Hindi. The reasons for these are as follows:
    a. The datasets we used are very vast and hence training them on our personal laptops was not only very time consuming but also used more memory than we are able to provide. While tackling this problem, we tried to reduce the dataset size however this led to the issue of not having enough training data and the model producing garbage values. We used applications like Google Colab that allowed us to leverage their GPU however the memory issue could still not be resolved.
    b. The Speech to text feature only takes 5 second clips of audio.
    c. Many of the models had very different requirements that made running the whole project simultaneously very difficult. In the future we hope to use applications like Docker to help with these issues.





Conclusion & Key Takeaways

1. This project allowed us to experience working with 3 different types of datasets and understand the issues that come with integrating a large project.
2. In hindsight, we may have been overly ambitious while choosing this project. Choosing 3 different datasets made us evaluate almost 9 different models. Although in normal circumstances this is a great way to learn, especially since it involves hands-on work, the time constraint on the project made it quite difficult for us to do all of these especially since the libraries and pre-built models used all had different dependencies that made it very hard to combine all the three.
3. Additionally the fact that these were all large datasets and the problem of machine translation requiring a good amount of training made it hard for us to test out different methods since each method in itself took 4+ hours of training each.